PERCI (Perceptive Cognitive Intelligence)

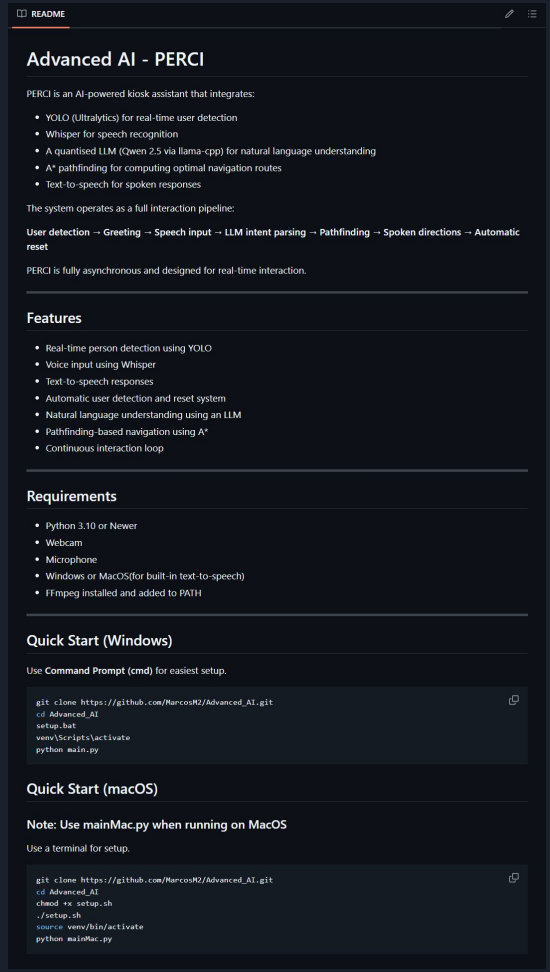
A Useful Assistant

What is PERCI?

- PERCI (Perceptive Cognitive Intelligence) is an AI system designed to help users navigate a building.
- Simply convert your building map to nodes and edges and PERCI will do the rest.
- To use PERCI walk up to it, allow it to introduce itself and tell it where you want to go. Its as simple a that!
- For this example we mapped out the ground and second floor of Áras an Phiarsaigh.

GitHub Repository

- Complete source code for the PERCI system.
- Automated environment setup using scripts.
- Creates venv automatically.
- Installs all required dependencies.
- Includes Quick start setup and installation guide.
- Cross Platform Windows, Mac and Linux.



README

Advanced AI - PERCI

PERCI is an AI-powered kiosk assistant that integrates:

- YOLO (Ultralytics) for real-time user detection
- Whisper for speech recognition
- A quantised LLM (Qwen 2.5 via llama-cpp) for natural language understanding
- A* pathfinding for computing optimal navigation routes
- Text-to-speech for spoken responses

The system operates as a full interaction pipeline:

User detection → Greeting → Speech input → LLM intent parsing → Pathfinding → Spoken directions → Automatic reset

PERCI is fully asynchronous and designed for real-time interaction.

Features

- Real-time person detection using YOLO
- Voice input using Whisper
- Text-to-speech responses
- Automatic user detection and reset system
- Natural language understanding using an LLM
- Pathfinding-based navigation using A*
- Continuous interaction loop

Requirements

- Python 3.10 or Newer
- Webcam
- Microphone
- Windows or MacOS(for built-in text-to-speech)
- FFmpeg installed and added to PATH

Quick Start (Windows)

Use **Command Prompt (cmd)** for easiest setup.

```
git clone https://github.com/MarcosM2/Advanced_AI.git
cd Advanced_AI
setup.bat
venv\Scripts\activate
python main.py
```

Quick Start (macOS)

Note: Use mainMac.py when running on MacOS

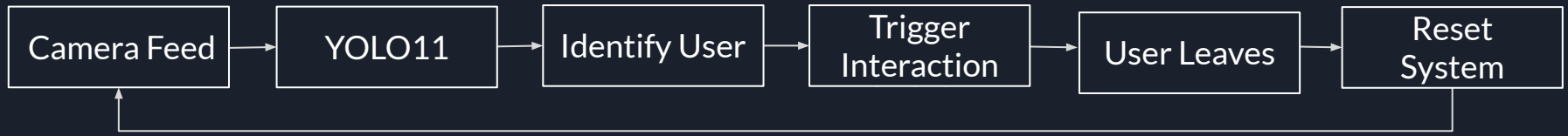
Use a terminal for setup.

```
git clone https://github.com/MarcosM2/Advanced_AI.git
cd Advanced_AI
chmod +x setup.sh
./setup.sh
source venv/bin/activate
python mainMac.py
```

The PERCI Pipeline

- User Detection → YOLO11 Nano Detects when a valid user is present or not.
- Text To Speech → Allows PERCI to interact with the user.
- Speech Recognition → Whisper base listens to user and transcribes speech for the LLM.
- LLM Processing → A quantised 4-bit version of Qwen 2.5 Instruct creates a response based on A star pathfinding algorithm and building map.

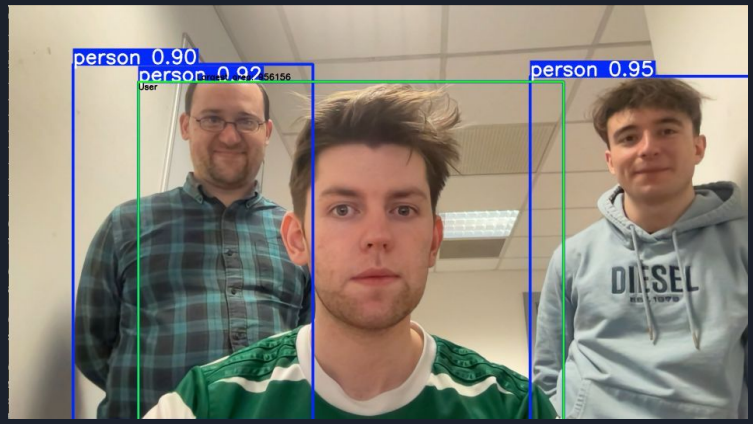
User Detection Pipeline



- Performs real time person detection from camera feed.
- Uses YOLO11 Nano object detection model.
- Identify if user is present or not.
- Triggers and ends user interaction respectively.

User Detection Overview

- Yolo 11 Nano, 2.6 million parameters, trained on COCO dataset.
- A user must be larger than 120,000 pixels squared.
- A valid user must be present for more than 3 seconds
- A user must be off screen for more than 2 seconds for the system to reset.
- If a user goes off screen for under 2 seconds they will still be considered a user.



Speech Recognition Pipeline

- Whisper converts spoken audio to text.
- Captures user input after system response.
- Outputs validated transcript for LLM processing.
- Operates asynchronously from the main thread to avoid blocking user detection.

Speech Recognition Overview

- Whisper Base = 74 million parameters.
- Encoder ≈ 23.7 million parameters.
- Decoder ≈ 48.9 million parameters.
- Background thread exits once transcription is complete.
- Records audio at 16 kHz from microphone.

```
[WHISPER] Listening...
[WHISPER] Transcribing...
[USER SAID] to the toilets.
```

LLM Processing Pipeline

- LLM takes in the transcribed user input from Whisper
- Determines whether the input is a request for directions around the building
- If so, extracts their desired destination from the request and passes it to pathfinding algorithms

LLM Processing Pipeline

- Retrieval-Augmented Generation (RAG) used to assist with parsing the request
  - There are only a limited number of possible destinations
  - The paths to them are more or less deterministic
  - We pass the list of possible destinations to the LLM as part of the user request
  - Our goal: Reduced hallucinations, more consistent results
  - Leverages strengths of LLM (better natural language processing), while mitigating its flaws (lack of intrinsic context)

LLM Processing Pipeline

- Model chosen: Qwen2.5-0.5B-Instruct-Q4_K_M.gguf, 500 million parameters
- Originally intended to use Qwen3-0.6B
  - Qwen series of LLMs designed by Alibaba Cloud
  - Available in a range of sizes from <1B parameters to >200B
  - Includes hybrid thinking modes, where developer can change from deep reasoning to quick responses
    - We decided against using the thinking mode; too slow (≅8-10x slower than non-thinking mode)
      - Instruct models turn the thinking mode off without needing to set this explicitly
    - Reason for using Qwen2.5 over Qwen3: Smaller Instruct models available
  - Open-source under the Apache 2.0 licence

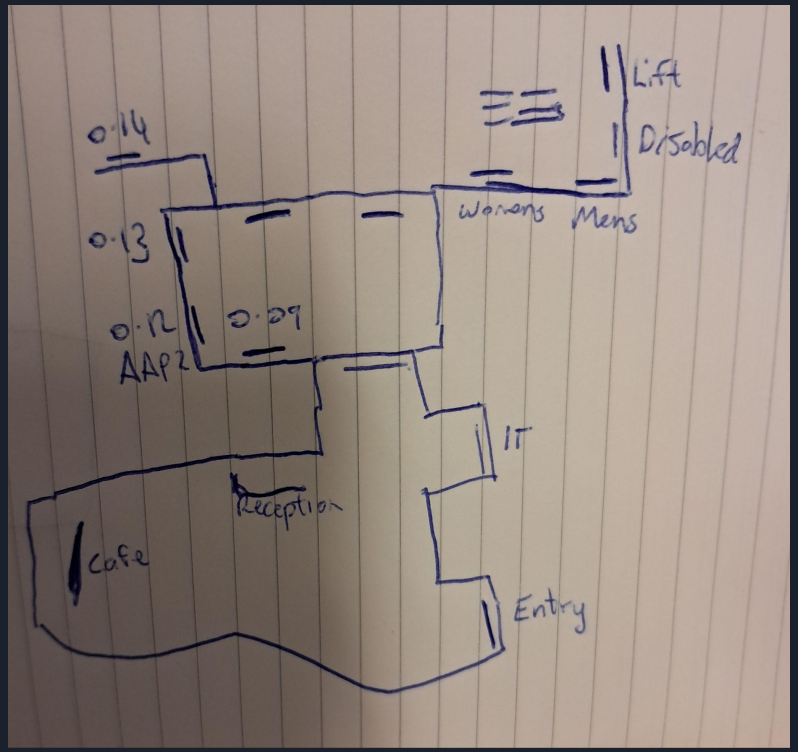
LLM Processing Pipeline

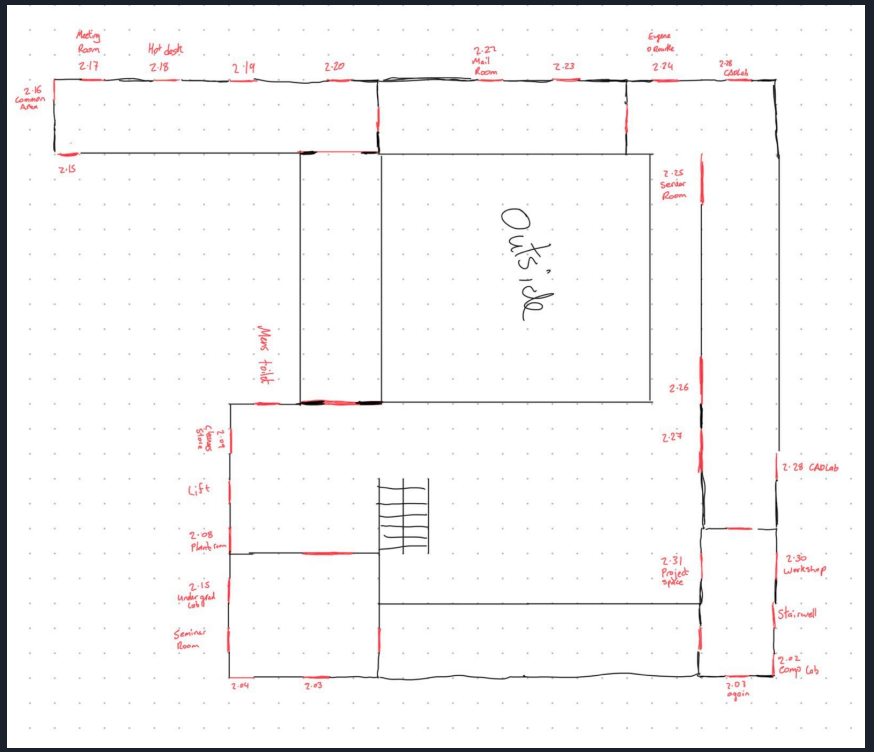
- Chosen model quantised using GGUF
- GGUF: GPT-Generated Unified Format
  - Developed by Georgi Gerganov, successor to GGML
  - De-facto standard for quantisation
    - Both official and unofficial Qwen implementations in GGUF format
  - Runs through llama-cpp, allowing faster inference through efficient, cross-platform C/C++ implementation requiring no further dependencies
  - Offers a variety of different quantisation types, from 2-bit to 8-bit

LLM Processing Pipeline

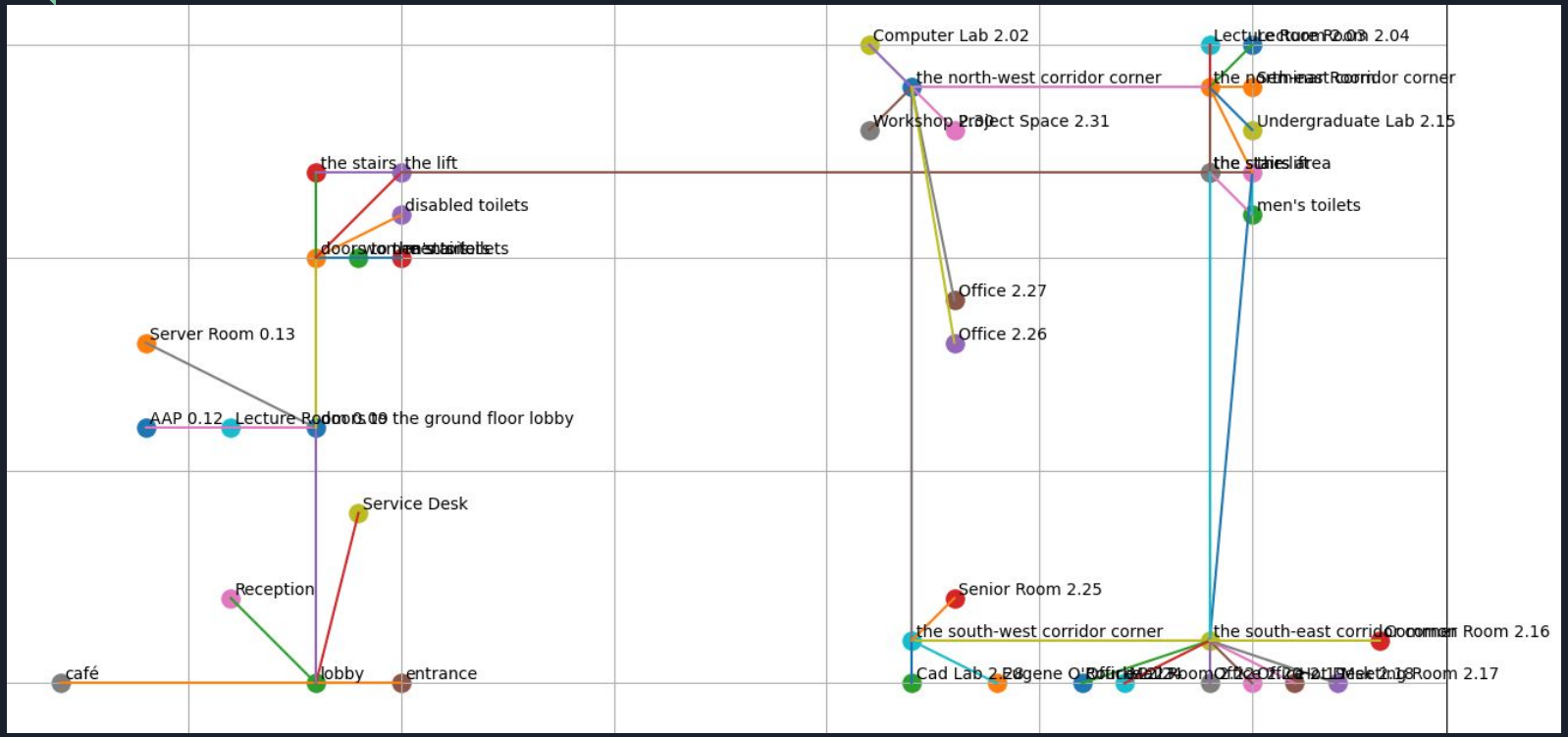
- Q4_K_M quantisation type used for the chosen model
  - Weights converted from 16-bit float to 4-bit integers
  - But not just converted naively
    - Scaling, grouping and encoding ranges used so that the original values can be reconstructed with higher accuracy
    - Weights w obtained from quants q and block minimum m using
      - `w = d * q + m`
  - Q4_K_M uses super-blocks containing 8 blocks of 32 weights; scales and minimums implemented using 6 bits - equates to 4.5 bits per weight
  - ≅ 70% reduction in size over original model

Dataset


Lift
Disabled
0.14
0.13
0.12
AAP2
0.09
Womens
Mens
IT
Reception
Cafe
Entry


Outside
Lift
2.08
2.15
2.25
2.26
2.27
2.28 CADLab
2.30
2.31
2.02
2.04
2.03

Dataset


Computer Lab 2.02
the north-west corridor corner
Workshop Project Space 2.31
the stairs
the lift
disabled toilets
Office 2.27
Office 2.26
Server Room 0.13
AAP 0.12
the ground floor lobby
Service Desk
Reception
café
lobby
entrance
Undergraduate Lab 2.15
men's toilets
Senior Room 2.25
the south-west corridor corner
Cad Lab 2.28
Room 2.17

Dataset

- Dataset comprises a series of nodes and edges
- Nodes either represent potential destinations, or intermediate points that can be used to reach destinations
    - Intermediate points chosen to provide more natural output to the user; more closely resemble how a human might give directions
    - Example intermediate points: Corners, crossroads, T-junctions
- Edges represent point-to-point connections between two nodes

Dataset

Node structure:

```
class Node:
    def __init__(self, node_id: str, name: str, x: float, y: float):
        self.id = node_id
        self.name = name
        self.x = x
        self.y = y
```

Example Node:

```
"entrance": Node("entrance", "Entrance", 0, 0),
```

- id: An identifier for the node for use with the pathfinding algorithm
- name: A human-readable description of the node
- x, y: x and y coordinates for the node in a grid - used in a heuristic in the pathfinding algorithm

Dataset

Example Edge:

```
("entrance", "lobby", 4, "From the entrance, walk forward into the lobby."),
```

- Structure:
  - origin: Defines the point which the user comes from
  - target: Defines the point which the user will go to on this edge
    - Can be terminal (destination reached) or non-terminal (just a step on the way to the destination)
  - weight: Manhattan/taxicab distance between the nodes
  - description: Describes, in human terms, how to get from the origin to the target
    - Descriptions are chained together once shortest path from origin to destination is defined and passed back to Whisper for text-to-speech output

LLM Logic

- LLM used as a Natural Language Processing (NLP) system
- User prompt converted to text and passed to the LLM along with the predefined list of possible destinations
- LLM prompted to extract details from user prompt
    - Whether or not the prompt is a request for directions
    - Whether the desired destination is clearly defined
    - What the destination is from the list

LLM Logic

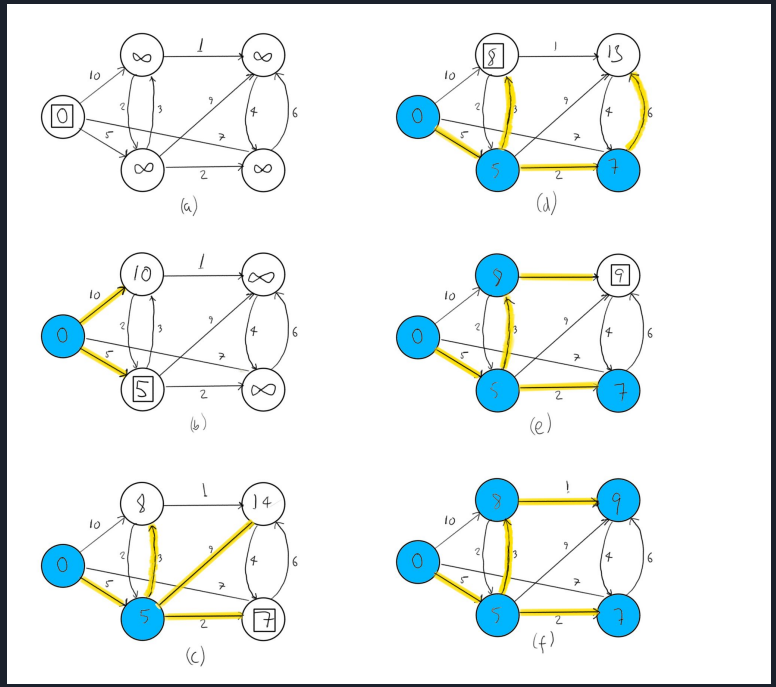
- Output of the LLM is a JSON object

```
{"intent": "navigation" | "not_navigation", "destination_id": string | null, "needs_clarification": boolean}
```

    - Provides a standard output template that can be further manipulated
    - Python has native libraries for processing JSON objects
- In the case that the LLM interprets the query as a valid request for navigation and a valid destination from the list is detected, the destination will be extracted from the JSON object and passed to the A* algorithm
- Otherwise, the LLM provides pre-programmed statements either stating that it can only help with directions, or that it needs clarification

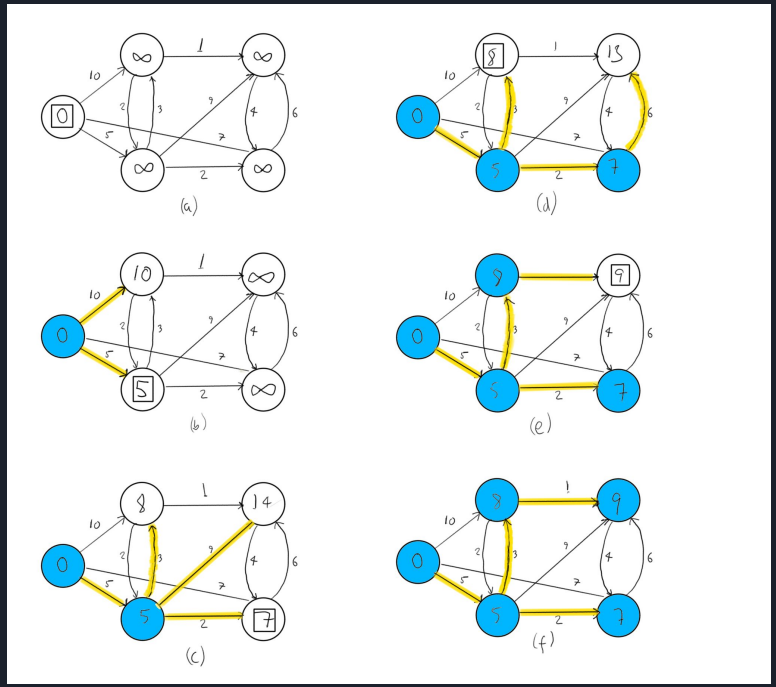
A* Pathfinding

- Pathfinding done using A* algorithm
  - Adaptation of Dijkstra's algorithm
  - Takes in an origin and destination node
    - Our origin is the entrance, but this algorithm isn't limited to any specific node
  - Outputs the shortest path between them, comprising a list of nodes that need to be passed between to get there



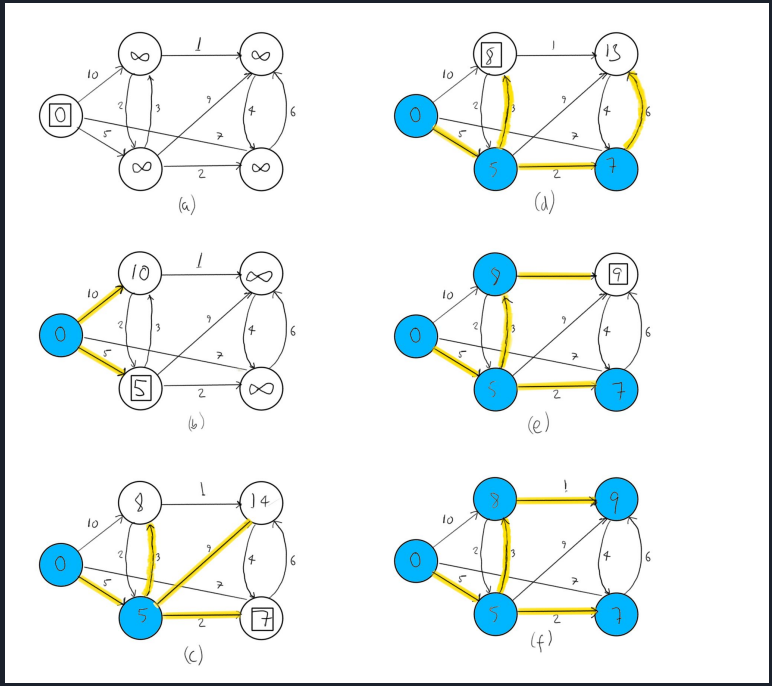
A* Pathfinding

- Dijkstra's algorithm
  - Developed by Edsger Dijkstra in 1956
  - Finds the shortest path from an origin node to all other nodes in a directed graph
  - Nodes connected by edges with non-negative weights
  - Maintains three groups of nodes:
    - Nodes not yet visited
    - Nodes visited, but shortest path not yet found
    - Nodes visited, where shortest path has been found



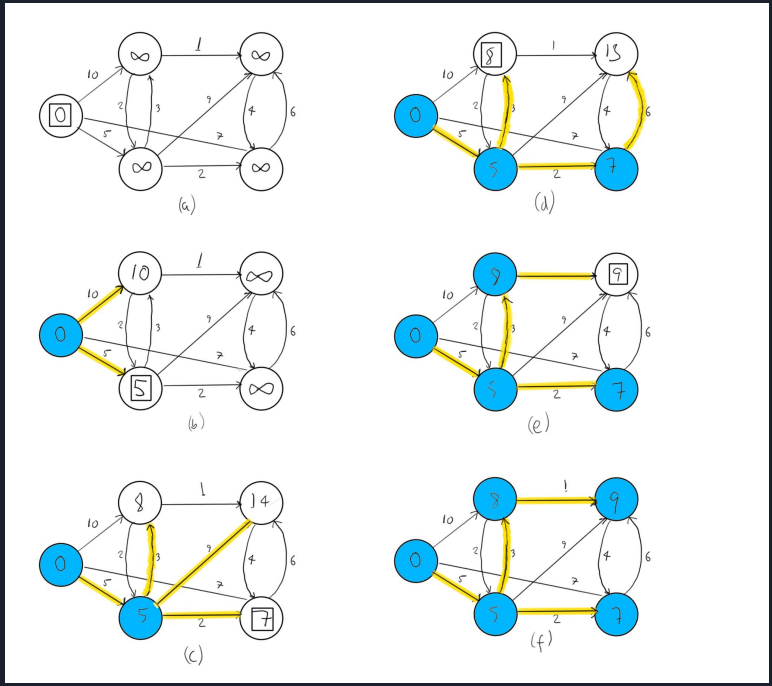
A* Pathfinding

- Dijkstra's algorithm
  - Starts by assigning a shortest-path estimate of 0 to the origin node and a cost of ∞ to all other nodes
  - Repeatedly selects the vertex with the minimum shortest-path estimate, adds this vertex to the list of nodes that have been visited and examines the edges leaving that vertex



A* Pathfinding

- Dijkstra's algorithm
  - If any paths to the nodes reachable from the vertex are less than the current estimate, a new estimate is set to the weight of the edge between the two nodes, plus the shortest-path estimate of the vertex being examined
  - The process is repeated until all nodes in the graph have been visited



A* Pathfinding

- A* search algorithm
  - Finds shortest path from origin node to a ***single*** destination node
  - Maintains a tree of nodes from the origin node, extending it one edge at a time until destination node is reached
  - At each iteration of the loop, A* determines which path to extend based on a combination of the current cost of the path and an estimate of the cost needed to extend the path to the destination node
    - Estimate is acquired using a heuristic function, e.g. the “as the crow flies” Euclidean distance to the goal from the current node
  - Once the destination node is reached, the full path is extracted using backtracking

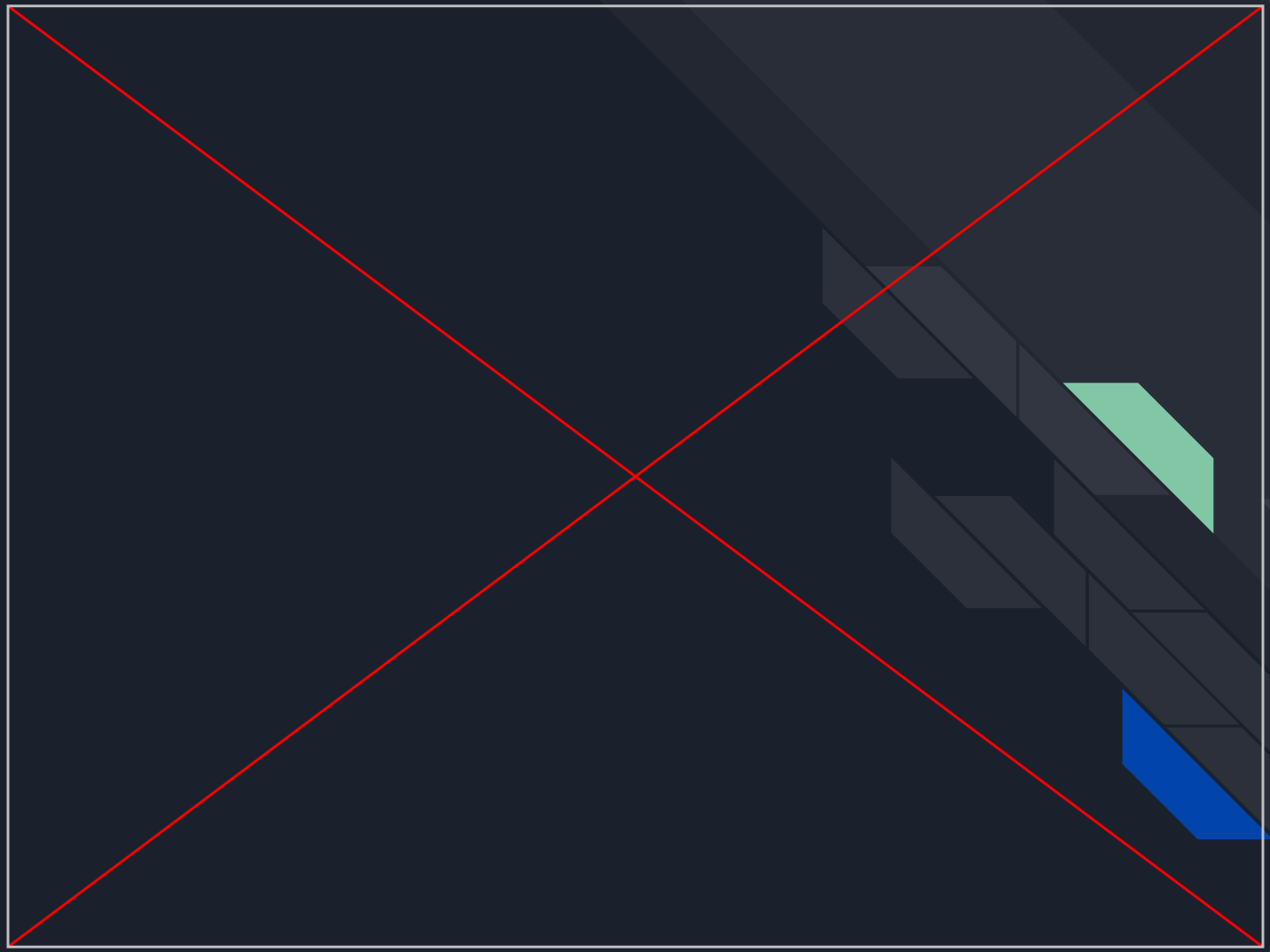
A* Pathfinding

- How do we deal with stairs and lifts?
  - Natural chokepoints - people have to use them to get to destinations on higher floors
  - However, we don’t want the algorithm passing through them unnecessarily
  - One solution:
    - Treat all floors on a map as part of the same 2D plane
    - Give edges representing stairs/lifts higher weights than any other edge on a given floor
    - Special case: If the user asks for stair-free access, temporarily set the stairs weights arbitrarily high - lift will always be preferred

Contributions

- Dataset assembled by the individual members of the team, Marcos, Eimhin and Richard
- Pipeline developed by the team:
  - Marcos: YOLO11 implementation
  - Eimhin: Whisper implementation
  - Richard: Qwen implementation
- YOLO reference paper: https://arxiv.org/abs/1506.02640
- Whisper reference paper: https://arxiv.org/abs/2212.04356
- Qwen2.5 technical report: https://arxiv.org/abs/2412.15115

Perci Video Demo



PERCI Live Demonstration